One thing you may notice is multiple calls to the `UtilitiesService`, which does things like clean fields to make them easier to index, convert dates from one system to another, and carry out shared complex functionality. I won't go into the specifics of the `UtilitiesService` in this appendix, so if you want you can check it out on GitHub.[1]

[1] If you use this link, github.com/cwhd/measurementor/blob/master/grails-app/services/org/cwhd/measure/UtilitiesService.groovy, you can get directly to the UtilitiesService class.

The patterns that you see in this service can be applied to a service that gets data from any other system. Coupled with a domain object that handles persistence and indexing, you have the pieces you need to get data from the source and analyze it in Kibana. The final piece is a set of jobs that update the data on a schedule.

B.1.4. Scheduling jobs for data collection

Normally I'm a big proponent of event-driven systems rather than timer-based systems. But because you're getting data from several source systems, each of which you may not have control over, it's easier for you to use timers to update and index the data.

The technology you'll use to schedule calling the data-collection services is Quartz. Quartz allows you to schedule jobs to run at any frequency you define. The Quartz plugin for Grails is flexible and allows you to define intervals using Cron (en.wikipedia.org/wiki/Cron), which is a standard UNIX convention, or use simple declarations in which you define the repeat interval in milliseconds.

About Quartz

Quartz is a flexible, lightweight, fault-tolerant job scheduler written in Java that's commonly used in applications that need scheduling capabilities. The code for Quartz is open source, so you can compile it on your own if you like, but it's more commonly used in its packaged form. Quartz is very popular and has been around for a long time, so the code is hardened and